

Feature Engineering Report

Author: Muhammad Saad, ARZ-2026-J4V9, AI, Automation & Security Engineering Advance Intern

Introduction

Security feature engineering transforms raw security data (logs, network flows, authentication records, etc.) into **informative features** that improve threat detection and analysis. It involves selecting, transforming, and constructing features that capture relevant behavior patterns [9†L159-L168] . In practice, analysts and data scientists use domain knowledge (e.g. known attack vectors and user behavior norms) to craft these features [6†L341-L349] . For example, one guide notes that feature engineering often “delves into selecting, transforming, and constructing features that are informative and relevant for identifying cyber threats” [9†L159-L168] . By incorporating expert insights, we can encode complex signals (like timing or sequence patterns) that help machine learning models detect anomalies [6†L327-L335] .

What is Security Feature Engineering?

Security feature engineering is the systematic process of extracting, transforming, and selecting data attributes to serve as inputs for security analytics (such as intrusion or anomaly detection). It combines **raw telemetry** (e.g. logs, packet captures, NetFlow data) and domain expertise to create metrics that distinguish normal from malicious activity. For example, features can capture statistical properties (means, variances), temporal patterns (login intervals), or categorical summaries (count of failed authentications). Incorporating *domain knowledge* is critical: security experts often define custom features tuned to specific threats (e.g. tracking unusual ports, login times, or command sequences) [6†L341-L349] . In short, security feature engineering means crafting a set of measurable indicators from raw security data that highlight deviations or anomalies linked to attacks [9†L159-L168] [6†L327-L335] .

Feature 1: Failed Login Count

- **Definition/Formula:** The *Failed Login Count* is the number of unsuccessful authentication attempts for a given user or account within a time window. Formally, if we denote each login event by e_i and let $I(e_i)$ indicate a failure (e.g. an event code for login failure), then:

$$\text{FailedLoginCount} = \sum_{i=1}^N I(e_i = \text{failure}).$$

It is simply the total count of events tagged as failed logins in the dataset.

- **Purpose:** This feature highlights brute-force or credential-guessing attacks. A sudden spike in failed logins for an account often signals that attackers are trying many passwords. For example, Splunk Enterprise Security uses spikes in failed login counts to detect brute-force attacks: if a user usually has ~5 failures per day but suddenly has 50 in one hour, the system flags an “excessive failed logins” anomaly [34†L55-L63]. Thus, *Failed Login Count* helps identify accounts under attack or suspicious login behavior.
- **Required Data Source:** Authentication or security logs (e.g. system event logs, SSH logs, RADIUS/LDAP logs) that record login success/failure events. In Windows, for example, Event ID 4625 tracks failed logins; other platforms record similar codes. We aggregate these log events for each user/account over the analysis period to compute the count.
- **Privacy Risk:** This feature involves user identifiers (usernames, account IDs) and possibly IP addresses. Both are **personally identifying information (PII)**. Storing or sharing raw counts tied to user IDs can reveal user behavior (e.g. how often a person attempts to log in) and potentially re-identify individuals. Even without usernames, an IP address associated with failures can be sensitive.
- **Mitigation:** To reduce risk, apply *pseudonymization* or hashing to identities in the logs. For example, replace each username or IP with a reversible token so that analysts can compute counts without seeing the actual ID [16†L383-L391]. Restrict access to the raw mapping key separately. Alternatively, drop or truncate IPs (e.g. keep only /24 subnet) if precise location is not needed. You can also add noise to the final count (differential privacy) when sharing aggregated results, or report counts only above a threshold to avoid singling out low-activity users. Overall, treat this feature as sensitive data and handle it like personal data under privacy regulations.

Feature 2: Unique Source IP Count

- **Definition/Formula:** *Unique Source IP Count* is the number of distinct source IP addresses seen contacting a host or service within a time window. Mathematically, if $\{IP_i\}$ are the source IPs of network sessions or login attempts in period T , then:

$$\text{UniqueIPCount} = |\{\text{unique } IP_i\}|.$$

It is simply the cardinality of the set of source IPs.

- **Purpose:** This feature measures **traffic or login diversity**. A very high unique-IP count over a short period can indicate distributed scanning or a botnet/DDoS attack. For instance, DDoS protection systems trigger alerts when thousands of distinct source IPs appear in a brief window [40†L209-L213]. By tracking unique IPs, we can detect coordinated attacks (many IPs hitting one target) or

reconnaissance (scans). Conversely, a low count (same IP repeatedly) could indicate a legitimate user or a targeted attacker focusing from one machine.

- **Required Data Source:** Network flow logs (NetFlow/IPFIX), firewall logs, or application logs that include source IP fields. We need logs of connections or authentication events with source IP addresses. The data should cover all relevant traffic to the monitored host/service.
- **Privacy Risk:** IP addresses are considered personal data under many privacy laws (they can often be traced to individuals or households). Publishing or retaining raw unique IP lists risks exposing user locations or identifying networked devices. Even the count could be sensitive if tied to a single server used by few people.
- **Mitigation:** Anonymize IP data by hashing or truncating. For example, convert each IP to a pseudonym in the dataset, or use only /24 subnet to reduce precision. When reporting unique counts, you can do it in aggregate form (e.g. “over 1000 sources”) to mask exact values. As with Feature 1, pseudonymization reduces direct identification [16†L383-L391]. In some settings, use k -anonymity by grouping IPs into geographic regions so that each bucket has $\geq k$ addresses, preventing singling out rare IPs [18†L101-L110]. Ensure only authorized systems can map back to original IPs (key protected separately).

Feature 3: Average Login Interval

- **Definition/Formula:** *Average Login Interval* measures the typical time gap between consecutive login attempts by a user. If a user has login timestamps $t_1, t_2, \dots, t_N$ in chronological order, the average interval $\bar{I}$ can be computed as:

$$\bar{I} = \frac{1}{N-1} \sum_{i=1}^{N-1} (t_{i+1} - t_i).$$

In practice, we convert timestamps to minutes or hours and average the differences.

- **Purpose:** This feature captures a user’s normal login frequency. Unusual behavior – such as extremely short intervals (many rapid logins) or very long gaps – may indicate automation or account compromise. For example, bots or credential stuffing often generate logins seconds apart, deviating from a human’s usual pattern. One study specifically engineered an “average login interval” feature from authentication logs to quantify user behavior [45†L414-L422]. By comparing a user’s current average interval to their baseline, we can detect anomalies (e.g. a user that typically logs in every 8 hours suddenly logs in every minute).
- **Required Data Source:** Authentication logs with precise timestamps. This could include system logs (e.g. Windows Security logs, Linux auth logs) or application

login records. We extract all successful login times for each user to compute intervals.

- **Privacy Risk:** This feature is based on timing information only. It does not directly reveal identity beyond the user account used (which is often already needed to group events). However, time-series patterns could indirectly fingerprint a user. The risk is lower here than directly storing location or content.
- **Mitigation:** Use the feature aggregated or relative to user baseline. If sharing data across users, avoid tagging which user produced which interval; you could use group statistics (e.g. “mean interval for group X”). If individual tracking is needed, ensure login timestamps are protected (e.g. encrypted) except in analysis. In general, since this is a derived numeric feature, apply differential privacy by adding small noise if publishing the results.

Feature 4: Geo Location Diversity

- **Definition/Formula:** *Geo Location Diversity* quantifies the variety of geographic regions (e.g. countries or cities) from which an account logs in. One simple measure is the number of unique geolocations over a period. For example, if a user’s last three logins came from USA, USA, and Germany, the count is 2. More formally:

$$\text{GeoDiversity} = |\{\text{unique countries or regions in login logs}\}|.$$

Alternately, one could use entropy over location distribution, but count of distinct locations is often sufficient.

- **Purpose:** This feature detects *impossible travel* or unusual location patterns. If a user normally logs in from one country and suddenly logs in from distant locations, that raises suspicion. Security platforms use this idea: Graylog’s “impossible travel” detector compares the geographic distance between a user’s current and prior login to see if the implied travel speed is unrealistic [43†L79-L84]. A high geo-diversity score (many countries in short time) flags that an account may be compromised or being used by multiple people.
- **Required Data Source:** Authentication logs with source IP addresses. We use a GeoIP database (e.g. MaxMind) to map each IP to a country or city. The diversity is then computed from those geolocations. If precise city-level data is too granular, country-level is often used for privacy.
- **Privacy Risk:** Geo data can reveal a user’s physical location or travel patterns, which is highly sensitive. Storing exact lat/long or city is risky. Even country-level location, if tied to a single user’s timeline, can indirectly identify them (e.g. if someone rarely travels, seeing two distant countries in logs is telling).

- **Mitigation:** Limit granularity – for example, only use country or time-zones instead of exact GPS coordinates. Alternatively, use *pseudonymization* of locations by mapping IPs to hashed region IDs so raw location isn’t stored. In risk scoring, you might only count “new location vs. known location” rather than exact values. Differential privacy can also help: for example, only flag geo-diversity anomalies without revealing raw locations (adding noise to location counts). As [43] suggests, focus on whether travel speed is implausible, not the actual coordinates 【43†L79-L84】 .

Feature 5: Session Duration

- **Definition/Formula:** *Session Duration* is the length of time a user’s session remains active (from login to logout or timeout). If a session starts at time t_{start} and ends at t_{end} , then

$$\text{SessionDuration} = t_{\text{end}} - t_{\text{start}}.$$

(If multiple sessions exist, one might take the average or maximum duration over a period.)

- **Purpose:** Abnormally long or short sessions can indicate abuse. For example, very long sessions (several hours or days) may point to an attacker establishing persistence (especially in remote access scenarios like RDP). Elastic Security describes a rule where unusually **high variance in remote session durations** (e.g. RDP connections lasting far longer than normal) triggers an alert, as adversaries may use prolonged sessions to evade detection or move laterally 【49†L19-L22】 . Conversely, extremely short sessions (instantly dropping out) could indicate automated probes. Tracking session length helps spot such anomalies relative to a user’s norm.
- **Required Data Source:** Audit or network logs that record login and logout times. For example, Windows security logs, VPN logs, or SSH session logs. Even network flow data (tracking start and end of flows per user) can be used to infer session length.
- **Privacy Risk:** Session timestamps and durations are not highly sensitive on their own, as they reveal only usage patterns (not the content). The main concern is linking them back to specific user IDs.
- **Mitigation:** Treat session durations as pseudonymous if possible – aggregate statistics across many users, or report percentile thresholds instead of exact values. If sharing durations, add minimal noise to prevent exact reconstruction (differential privacy). Otherwise, standard access controls (keeping logs in secure

SIEM systems) are typically sufficient, as this feature is usually not considered PII per se.

Feature 6: Bytes Sent

- **Definition/Formula:** *Bytes Sent* is the total amount of data transmitted *outbound* from a host (or client) during a session or time window. If b_i is the size of packet i sent by the user, then

$$\text{BytesSent} = \sum_i b_i.$$

We typically measure this in kilobytes or megabytes per session or per hour.

- **Purpose:** This feature helps detect data exfiltration or anomalous upload activity. A user suddenly sending large volumes of data (e.g. gigabytes) may indicate bulk data theft or malware communicating. In one anomaly example, if a server normally sends ~1 GB/day and abruptly spikes to 50 GB, that is flagged as an “extreme anomaly” likely due to exfiltration or heavy misuse [34†L72-L79]. Thus, *Bytes Sent* can signal unusual activity patterns (especially when baselined against the user’s normal behavior or cohort baseline).
- **Required Data Source:** Network flow logs (NetFlow/IPFIX), firewall logs, or any system that records byte counts per connection/session. For web or application servers, web logs or proxy logs often include bytes transferred.
- **Privacy Risk:** The raw count of bytes isn’t PII, but it can reveal user habits (e.g. if someone streams video or downloads large files regularly). In aggregated form, it’s usually acceptable. The main risk is if combined with other features to profile a user uniquely.
- **Mitigation:** When sharing this feature, aggregate or discretize the values (e.g. low/medium/high) or add slight noise. You could also normalize by expected baselines. Since *Bytes Sent* is a volume metric, anonymization typically focuses on ensuring no content is included (only counts). Encryption in logs and role-based access are practical mitigations, along with possible differential privacy if publishing statistics.

Feature 7: Bytes Received

- **Definition/Formula:** *Bytes Received* is the total amount of data transferred *inbound* to a host during a session or window. Formally:

$$\text{BytesReceived} = \sum_i r_i,$$

where r_i are the sizes of incoming packets.

- **Purpose:** This feature can indicate large downloads or inbound traffic surges. For example, a host suddenly receiving a flood of traffic (tens of gigabytes) may be under a DDoS attack or receiving a large payload (such as during data exfiltration from a victim to an attacker-controlled server via pull). Monitoring inbound bytes can thus complement *Bytes Sent* in spotting unusual spikes.
- **Required Data Source:** Like *Bytes Sent*, this uses network flow or firewall logs that separately count inbound bytes. It may require distinguishing source vs. destination in the flows (or separate in/out fields in NetFlow).
- **Privacy Risk:** Low – it records only the volume, not content or source identity (assuming the source IP is handled separately). However, if correlated with specific sources, there could be some profiling risk (e.g. download habits).
- **Mitigation:** Similar to *Bytes Sent*. Usually, focus on aggregated anomalies. If publishing, add noise or bucket values. Ensure flows are anonymized if sharing beyond trusted analysts.

Feature 8: Port Diversity

- **Definition/Formula:** *Port Diversity* measures how many different network ports a host or client uses. A common metric is the Shannon entropy of port numbers in the session(s):

$$H_{\text{ports}} = - \sum_p P(p) \log_2 P(p),$$

where $P(p)$ is the proportion of traffic on port p . High entropy indicates connections are spread over many ports. Another simpler measure is just the count of distinct destination ports.

- **Purpose:** This captures unusual scanning or connection behavior. For example, a client connecting to hundreds of different ports on a server is likely scanning. Conversely, a compromised server opening many outbound ports may be probing the network. Studies have found port-entropy to be a useful intrusion-detection feature: high port-diversity often correlates with reconnaissance or malware spreading. In one model, entropy over destination ports (H_{dport}) was explicitly defined as a feature (see formula above) [54†L531-L534] .
- **Required Data Source:** Network session or flow logs including destination port fields. Both client and server logs can be used (e.g. proxy logs, firewall logs) to capture which ports were used.

- **Privacy Risk:** None in terms of user identity, since port numbers are not personal data. However, the usage pattern could reveal a user's installed applications or targeted services.
- **Mitigation:** Usually not needed for privacy, but be mindful: if a user's specific port usage habits are sensitive, you could simply report relative measures or ranges. The entropy formula itself already abstracts details. Standard security of logs applies.

Feature 9: Failed Authentication Ratio

- **Definition/Formula:** This is the ratio of failed logins to total login attempts. If F = number of failed logins and S = number of successful logins in a period, then:

$$\text{FailedAuthRatio} = \frac{F}{F+S}.$$

It's a unitless fraction between 0 (no failures) and 1 (all attempts fail).

- **Purpose:** While *Failed Login Count* tracks volume of failures, the ratio normalizes by activity level. An account with a 90% failure ratio is suspicious (e.g. under attack), even if total attempts are low. Conversely, 2 failures out of 100 attempts (2%) might be normal. This feature helps distinguish trivial failures from systematic issues.
- **Required Data Source:** Same authentication logs as for Feature 1, capturing both successes and failures. We compute the counts of each event type per user.
- **Privacy Risk:** Same as Feature 1: involves usernames and login events. It indirectly encodes user activity level.
- **Mitigation:** Use the same measures as for failed counts: pseudonymize or hash user IDs, and perhaps only report the ratio without user context. Aggregating over a group of accounts can also protect individuals. Differential privacy can add noise to the ratio if the total number of events is low.

Feature 10: Threat Score

- **Definition:** *Threat Score* is a composite metric that summarizes multiple feature anomalies into one risk value. It is not a raw data feature, but the output of a scoring model or formula that weighs the above features. For example, a system might combine deviations in failed login count, geolocation, and session duration into a single score.
- **Purpose:** The threat score provides an overall indicator of an account's or host's risk level. Instead of inspecting each feature separately, analysts can monitor if the combined score crosses a threshold. For example, an anomaly-detection system

might compute a dynamic risk score per user: “each user is assigned a risk score based on deviations from their established behavioral baseline” [45†L432-L438] . This alerts on users whose aggregate behavior (e.g. many features out of norm) is dangerously abnormal.

- **Required Data Source:** All of the above features and possibly more. The system needs the engineered features in real time. The threat score itself is produced by a formula or machine learning model (e.g. weighted sum or anomaly detector) trained on normal vs. malicious patterns.
- **Privacy Risk:** The risk score itself is a dimensionless index, but it is derived from potentially sensitive inputs. If scores are stored or shared, they could leak that a specific user is under investigation. The score should be treated as sensitive insights.
- **Mitigation:** Keep the scoring algorithm internal to the security system; do not expose how it is calculated. When communicating risk, refer to the score qualitatively (e.g. “high risk”) without exposing numeric values tied to individuals. Ensure that the input features have already been privacy-processed (as above). In some cases, one can add small random perturbation to final scores to prevent exact profiling, but clarity to analysts must be balanced.

Privacy-Aware Feature Engineering

When engineering the above features, we must embed privacy protections into the process itself. Key techniques include **pseudonymization**, **k-anonymity**, and **differential privacy**:

- **Pseudonymization:** Replace direct identifiers (usernames, IPs) with *pseudonyms* or tokens. This means that data like “user123” becomes “A1B2C3”, and only a separate secure lookup can revert it. Pseudonymization allows analysis on the data (e.g. counting logins per pseudonym) without exposing real identities [16†L383-L391] . It preserves referential integrity (analysis results still map 1-to-1 with individuals) but requires a protected key to reverse. As Imperva notes, pseudonymization maintains data utility while masking direct identifiers [16†L383-L391] . It is reversible, so the key must be tightly secured (unlike full anonymization).
- **k-Anonymity:** Ensure that each individual’s record is indistinguishable from at least $(k-1)$ others with respect to quasi-identifiers. For example, group login records so that at least k users share the same combination of features (e.g. same IP region and time bucket). In practice, one could bucket features (age, location, device type) so that no user is unique in the dataset. K-anonymity “prevents PII from being disclosed” by masking or generalizing identifiers [18†L101-L110] . For instance, only report that *at least 5 users had 10+ failed logins*, rather than which user. This reduces re-identification risk when sharing data.

- **Differential Privacy:** Add carefully calibrated noise to aggregates or feature values so that results do not reveal individual data. The core guarantee is that the removal or addition of any single user's data has minimal impact on the output. In anomaly features, this could mean perturbing counts or scores. Differential privacy protects individuals in large datasets: as IEEE explains, it ensures that “the removal of a single data point cannot alter the overall data” beyond a small amount [21†L111-L119]. For example, when reporting global statistics (like average session length), adding random noise prevents inferring any one session. Differential privacy is especially useful when publishing data or sharing feature sets externally.

Each of these methods helps balance privacy with the utility of the security features. In a professional report, we would document that identifiers were hashed, thresholds applied, and noise added so that no person's exact behavior is exposed.

Feature Selection

After feature engineering, **feature selection** is used to choose the most relevant subset of features for a model. Feature selection improves performance by eliminating redundant or irrelevant attributes [23†L231-L239]. In security analytics, this means picking those features (among the above or others) that best distinguish attacks. For example, if bytes transferred is uncorrelated with threats in a given context, it may be dropped.

As IBM notes, feature selection “reduces the feature space to a selected subset, improving model performance while lowering computational demands” [23†L231-L239]. The benefits include faster training, fewer false positives (less overfitting), and easier interpretability. In privacy terms, feature selection can also reduce risk: by dropping sensitive features that add little value, we minimize exposure of PII. For instance, if geolocation can be replaced by a simpler ‘impossible travel’ indicator, we need not store raw coordinates. Ultimately, systematic feature selection (e.g. via mutual information or Lasso regularization) should be performed on the engineered features to optimize the security model.

Conclusion

In summary, this report defined ten key security features (Failed Login Count, Unique IP Count, Average Login Interval, Geo Diversity, Session Duration, Bytes Sent/Received, Port Diversity, Failed Auth Ratio, and an aggregate Threat Score) along with their formulas, purposes, data sources, and privacy considerations. Each feature helps expose different angles of anomalous behavior (e.g. brute-force attacks, scans, lateral movement, exfiltration). We also discussed how to engineer these features in a **privacy-aware** manner—using techniques like pseudonymization, *k*-anonymity, and differential privacy—to protect individuals' data. Finally, we noted that careful feature selection is necessary to build effective models without undue complexity.

Overall, a professional feature engineering report of this sort would be about 8–12 pages, include a cover page with the company logo and author details, a table of contents, and well-formatted sections as above. It would present formulas, diagrams (if applicable), and citations to industry and research sources to back up each technique 【34†L55-L63】 【23†L231-L239】 . This structure ensures clarity for readers and rigor in demonstrating how each security feature is constructed and used in analysis.

Sources: The definitions, purposes, and privacy guidance above draw on current cybersecurity research and industry materials 【34†L55-L63】 【40†L209-L213】 【43†L79-L84】 【45†L414-L422】 【49†L19-L22】 【54†L531-L534】 【16†L383-L391】 【18†L101-L110】 【21†L111-L119】 【23†L231-L239】 .
